

## 3. Guidance

Writing a Software Requirements Specification (SRS) is a fundamental step in the software development lifecycle. The SRS provides a clear, standardized, and structured definition of what the software must accomplish to meet user and system goals. It emphasizes functionality, behavior, and constraints while acting as a bridge between stakeholders, developers, and testers. An SRS also serves as the authoritative reference for all subsequent development and lifecycle processes, which underscores its criticality. The following guidance incorporates best practices, industry standards, and lessons learned to ensure clarity, completeness, adaptability, and traceability in software projects—particularly in complex, safety-critical environments.

When writing the SRS, it is important to capture specific, key information. Detailed guidance for capturing that information is provided in the sections below. Other sections, not listed below, that may be included in an SRS are:

- Audience, organization and contents of the SRS.
- Definitions, acronyms, abbreviations specific to the SRS.
- Summary of history of system development, operation, maintenance.
- Project sponsor, acquirer, user, developer, and support agencies.
- Current and planned operating sites.
- References.
- Appendices for supporting information like mockups, data models, or glossary.

Because requirements change and evolve over the life of the project, the SRS is typically revised as the project progresses. Guidance for managing these changes can be found in [SWE-053 - Manage Requirements Changes](#) of this Handbook. Guidance for baselining and updating the SRS in preparation for life cycle milestone reviews can be found in [Topic 7.08 - Maturity of Life Cycle Products at Milestone Reviews](#) in this Handbook. Data on requirements volatility is tracked in the Software Metrics Report (see [SWE-200 - Software Requirements Volatility Metrics](#) for additional information).

The following sections provide detailed guidance for implementing the Minimum Recommended Content of an SRS, highlighting examples, tips, and best practices for writing effective requirements.

### 3.1 Introduction

The introductory section provides the foundational context for the SRS and provides essential background information. Use it to establish the purpose and scope of the software requirements and introduce the software system being built.

#### 3.1.1 Purpose

Specify the purpose of the SRS and how it contributes to the project's lifecycle and stakeholders. Identify the document's audience—such as developers, testers, system engineers, users, and acquirers.

Example: "The purpose of this SRS is to define the software requirements for the XYZ Mission Control Software. The SRS serves as a blueprint for all development stages and a contractual reference for alignment between stakeholders."

#### 3.1.2 Scope

Describe the software product under development, including the scope of its goals, functions, and limitations. Highlight its relationship with the larger system or mission objectives. Discuss the problem domain and how the software addresses specific user needs.

Example: "The software will allow ground controllers to monitor spacecraft telemetry in real-time and send corrective commands. The system will operate with a maximum latency of 5 seconds under nominal conditions but will not perform onboard diagnostics, as these are managed at the subsystem level."

#### 3.1.3 Software System Overview

Provide an overview of the software system and its major components, including a high-level description of its functionality, context, and relationships with other systems or environments. The overview should allow the readers to quickly understand the software's purpose and architecture before diving into the specific requirements. It should include:

- **CSCI Identifier:** CSCI name or other identifier (e.g., CSCI name and abbreviation/acronym)
- **Key Functions/Capabilities:** High-level summary of primary functions and components
  - **Product Perspective:** How the software fits into the larger system.
  - **Product Functions:** What the software will do.
- **Background Information:** Include supporting background information e.g.:
  - **Operating Environment:** Supported hardware, software, and networks.
  - **User Classes and Characteristics:** Define primary (e.g., mission operators) and secondary (e.g., maintenance personnel) user groups, highlighting their unique needs.
  - **Assumptions and Dependencies:** External factors, constraints, or limitations that influence requirements.
  - **Design and Implementation Constraints:** Any limitations (e.g., programming languages, frameworks)
- **Interfaces:** High-level summary of primary user interfaces, hardware interfaces, software interfaces, communication interfaces. (Note when software design interface descriptions will be captured in a separate [5.02 - IDD - Interface Design Description](#), [5.13 - SwDD - Software Design Description](#), or Interface Requirements Specification/Document (IRS/IRD).)

Example: "The software will run on Red Hat Enterprise Linux 8.4 and interact with satellite payloads through the SpaceWire communication bus. It assumes ground station connectivity and relies on an external API for mission planning schedules."

## 3.2 CSCI Requirements

For purposes including, but not limited to, planning work, assigning resources, and understanding the size of the software, it is helpful to organize requirements by CSCIs. It is also helpful to understand the rationale behind a requirement to understand its intent; therefore, consider co-locating the "supporting requirements rationale", as described in Section 3.4 of this guidance, with each requirement.

The SRS compiles both functional and non-functional requirements along with interface and user requirements to ensure completeness, usability, and alignment with user and technical needs. Each requirement should satisfy these characteristics: specificity, traceability, and testability.

For each CSCI, include:

- Functional Requirements
- Non-functional Requirements
- Interface Requirements
- User Requirements

### 3.2.1 Functional Requirements

Functional requirements describe the specific behaviors, functions, and features a software system must perform — they define the core functions, features, and behaviors of the software from the user's perspective. These are the actions the system must perform to fulfill its intended purpose. Functional requirements are typically high-level and user focused and form the starting point for further analysis and refinement.

Identify Functional Requirements to:

- Translate user needs into technical specifications.
- Define the scope of the system's functionality.
- Guide developers in implementing features including:
  - Descriptions of system behaviors under specific conditions.
  - Descriptions of inputs, outputs, and processing logic.
  - Specifications for user interactions, data handling, and system responses.
- Serve as a basis for test case development and requirements traceability matrices to ensure traceability from business goals to system behaviors.

Tips for Writing Good Functional Requirements:

The following Best Practices should be implemented for each type of Functional Requirement in this section:

- Use clear and consistent language: Avoid ambiguity; use "shall" to indicate mandatory behavior.
- Follow a structured format: Include requirement ID, description, rationale, and acceptance criteria.
  - Number each requirement for easy reference (e.g., FR-1, FR-2).
- Include use cases or user stories: Provide context for how the requirement supports user goals.
- Define inputs and outputs: Specify data types, formats, validation rules, and conditions including:
  - Validity checks on the inputs. <sup>214</sup>
  - Relationship of outputs to inputs, including: <sup>214</sup>
    - Input/output sequences.
    - Formulas for input to output conversion.
- Include exception and error handling: Describe how the system should behave under off-nominal situations including: <sup>214</sup>
  - Overflow.
  - Communication facilities.
  - Error handling and recovery.
- Relevant operational modes (nominal, critical, contingency).
- Exact sequence of operations. <sup>214</sup>
- Effect of parameters. <sup>214</sup>
- Ensure testability: Write requirements that can be verified through testing.
- Prioritize requirements: Identify which functions are critical vs. optional.

Characteristics of Functional Requirements

- Specific: Clear and unambiguous (e.g., avoid subjective terms like "efficient" or "user-friendly")
- User-focused: They describe how the system will respond to user inputs.
- Specific and verifiable/testable: Each requirement should be clear enough to be tested with defined success criteria.
- Behavioral: They define system behavior or operations under specific conditions.
- Traceable: Linked to user needs, business goals, or use cases.
- Complete: Cover all expected inputs, outputs, and scenarios.

Common Types of Functional Requirements

- Authentication & Authorization: Role-specific access control Login, roles, permissions.
- Data Management: Create, read, update, delete (CRUD) operations.
- I/O Management: User interface and device responsiveness.
- Safety Features: Define emergency response triggers.
- Control Logic: How the system reacts to specific events or sequences.

- State Management: Support for operational modes like Idle, Active, or Contingency.
- Business Rules: Logic that governs how data is processed.
- Notifications: Emails, alerts, messages.
- Reporting: Generating and exporting reports.
- User Interfaces: What users can do on each screen or page.

Examples of Functional Requirements

| ID    | Requirement Example                                                                                               |
|-------|-------------------------------------------------------------------------------------------------------------------|
| FR-1  | <i>The system shall allow users to log in using a username and password.</i>                                      |
| FR-2  | <i>The system shall send a confirmation email after successful registration.</i>                                  |
| FR-3  | <i>The system shall allow users to search for products by name or category.</i>                                   |
| FR-4  | <i>The system shall generate a monthly sales report in PDF format.</i>                                            |
| FR-13 | <i>The system shall support retrieval of historical mission data within 3 seconds for up to 1 million records</i> |

Because functional requirements tend to be large in number, it may be helpful to organize them into groups or subsections appropriate for the software project. Include requirements for the following.

### 3.2.1.1 General Functional Requirements

The general functional requirements need to be established to form a starting point for further analysis and refinement. Performing software requirements analysis ensures that the software component of a system fully aligns with top-level system requirements, safety analyses, and hardware constraints. It promotes clarity, completeness, and quality, reducing risks of integration failures, system anomalies, and downstream costs caused by requirement misalignment. Asking the following questions may be helpful when identifying these requirements:

- What functions need to be performed?
- Where do they need to be performed?
- How often? (also useful for performance and timing requirements)
- Under what operational and environmental conditions? (also useful for environmental requirements) <sup>273</sup>

As systems become more complex, general functional requirements are often decomposed into smaller, more manageable sub-requirements. This process is known as requirements decomposition. Some of these requirements may not be explicitly stated but inferred. These are referred to as derived requirements. Both the decomposed and derived requirements need to be captured as part of the functional requirements. Conversations with Stakeholders will typically help decompose the higher-level functional requirements and expose any derived requirements.

 **Note:** Additional requirements may be identified during the Design phase. These design requirements should be added to the SRS as they are understood and analyzed. The updated SRS should be included as part of the PDR and/or CDR review packages to allow the Stakeholders to validate the requirements changes.

Expand the items below for additional information on decomposed and derived requirements.

#### 3.2.1.1.1 Decomposed Requirements

Decomposed requirements are lower-level, more detailed or refined requirements that result from breaking down the high-level functional or non-functional requirements into smaller, manageable components. They are typically listed as child requirements under a parent requirement, often using hierarchical IDs (e.g., REQ-001.1, REQ-001.2).

Think of decomposition as “breaking down” a requirement into its constituent parts.

Decompose requirements to:

- To clarify and detail what each functional requirement entails.
- To assign responsibilities to specific components or modules.
- To facilitate design, implementation, and testing.

Examples of Decomposed requirement

- Functional Requirement:
  - The system shall authenticate users.
- Decomposed Requirements:
  - The system shall validate the username format.
  - The system shall verify the password against the stored hash.
  - The system shall lock the account after 5 failed attempts.

These decomposed requirements are directly traceable to the parent functional requirement and help ensure completeness and testability.

#### 3.2.1.1.2 Derived Requirements

Derived requirements are requirements that are not explicitly stated by stakeholders but are inferred or generated from higher-level requirements, system constraints, or design decisions. They are necessary to support, enable, or ensure the fulfillment of other requirements. They are newly created requirements that are not directly part of the original

requirements decomposition but are essential for realizing the system's intended behavior. Derived Requirements are documented with a traceability link to the source requirement or rationale, often flagged as “derived” as the requirement type.

Think of derived requirements as “emerging needs” that arise from analysis or design.

Derive requirements to:

- To support or enable the fulfillment of functional requirements.
- To address technical constraints, such as performance, security, or compliance.
- To ensure system integrity in real-world operating conditions.

Examples of Derived requirements

- Functional Requirement:
  - The system shall allow users to reset their password.
- Derived Requirements:
  - The system shall send a one-time password (OTP) to the user’s registered email.
  - The system shall expire the OTP after 10 minutes.
  - The system shall log all password reset attempts for auditing.
  - *The password shall be 12 to 32 characters long in length and consist of at least one upper case character, one lower case character, a number, and special characters.*
  - *The password shall not be updated more than once in a 24-hour period.*

These are not directly requested by users but are necessary to implement the functional requirements securely and effectively.

3.2.1.1.3      **Summary of Relationships**

| Type       | Origin                                           | Purpose                                    | Traceability                     |
|------------|--------------------------------------------------|--------------------------------------------|----------------------------------|
| Functional | Directly from stakeholders                       | Define what the system must do             | High-level                       |
| Decomposed | From functional requirements                     | Break down into detailed, actionable parts | Directly traceable to functional |
| Derived    | Inferred from functional, constraints, or design | Support or enable functional requirements  | Indirectly traceable             |

3.2.1.1.4      **Best Practices**

- Use requirements traceability matrices (RTMs) to link functional, decomposed, and derived requirements.
- Validate derived requirements with stakeholders to ensure alignment.
- Keep decomposition logical and hierarchical to maintain clarity.
- Ensure all derived and decomposed requirements are testable and measurable.

See [SWE-050 - Software Requirements](#) and [SWE-051 - Software Requirements Analysis](#).

3.2.1.2      **State and Mode Requirements**

If the software is required to operate in more than one state or mode, having requirements that distinguish each state or mode is a necessity. Identify and define each state and mode. Each requirement or group of requirements should be correlated to the states and modes. The correlation may be documented in a table, an appendix, or by annotation of the requirements.

-  State requirements define the behavior of a system based on its current condition or status (i.e., its state). A state represents a specific situation during the life of a system where certain conditions hold true. They specify how the system should behave in each state and how it should transition between states based on events or conditions. State requirements are used in systems with finite state machines, event-driven architectures, or workflow-based logic. There should be clearly define entry/exit conditions for each state.
-  Mode requirements define the system’s behavior based on its operational context or configuration, often representing different levels of functionality or control. They specify how the system should behave in different modes of operation, such as normal, degraded, maintenance, or emergency. They are commonly used in safety-critical systems.

The differences between State and Mode requirements are illustrated below:

State vs. Mode: Key Differences

| Aspect     | State                           | Mode                                  |
|------------|---------------------------------|---------------------------------------|
| Definition | Current condition of the system | Operational context or configuration  |
| Scope      | Often short-term and reactive   | Often long-term and strategic         |
| Trigger    | Events or conditions            | Commands, faults, or system decisions |
| Example    | Idle, Active, Error             | Normal, Maintenance, Emergency        |

Examples of states and modes include idle, ready, active, post-use analysis, training, degraded, emergency, backup, launch, testing, and deployment. To put these into context, the table shows an example of each.

| Example of State Requirements                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Example of Mode Requirements                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Elevator Control System</p> <ul style="list-style-type: none"><li>State: Idle<br/>When the elevator is in the Idle state and receives a floor request, it shall transition to the Moving state.</li><li>State: Moving<br/>While in the Moving state, the elevator shall ignore new requests for the current direction until it reaches the requested floor.</li><li>State: Door Open<br/>The elevator shall remain in the Door Open state for 10 seconds before transitioning to Idle, unless a door obstruction is detected.</li></ul> | <p>Aircraft Flight Control System</p> <ul style="list-style-type: none"><li>Mode: Normal Flight Mode<br/>In Normal Flight Mode, the system shall allow full autopilot control and respond to pilot inputs for navigation and altitude adjustments.</li><li>Mode: Emergency Mode<br/>In Emergency Mode, the system shall disable autopilot and prioritize manual control inputs from the pilot.</li><li>Mode: Maintenance Mode<br/>In Maintenance Mode, the system shall allow access to diagnostic tools and inhibit flight control operations.</li></ul> |

3.2.1.3 Internal and External Data Requirements

Most software has internal or external data requirements for the system being designed and built. They detail key data definitions, formats, and storage requirements. As such, these requirements need to be identified and documented.

a. **Internal data requirements** specify the data that is created, stored, processed, or managed entirely within the system. This data is typically generated by the system itself or entered by users and is not dependent on external sources. It defines how the system handles its own data assets, such as files, databases, user profiles, logs, configurations, or transaction records.

Internal data requirements include information such as:

- Data types.
- Modes of access, e.g., random, sequential.
- Size and format.
- Units of measure.

Common types of internal data are:

- User accounts and credentials
- Application logs
- Configuration settings
- Audit trails
- Locally generated reports

Examples of internal data requirements are:

- *The system shall store user login history for 90 days.*
- *The application shall maintain a local cache of the last 100 search queries.*
- *The system shall encrypt all stored passwords using SHA-256.*

b. **External data requirements** specify the data that the system must access, retrieve, or exchange with external sources, such as third-party systems, databases, APIs, or services. It defines how the system interacts with external data providers, ensuring proper integration, data format compatibility, and security.

Common sources of external data are:

- APIs (e.g., payment gateways, weather services)
- External databases or data warehouses
- File imports (e.g., CSV, XML)
- Cloud services (e.g., AWS S3, Google Sheets)

Examples of external data requirements are:

- *The system shall retrieve weather data from the National Weather Service API every 15 minutes.*
- *The application shall import customer records from the external CRM system in CSV format.*
- *The system shall validate shipping addresses using a third-party geolocation service.*

Key Differences between Internal and External Data Requirements:

| Aspect   | Internal Data                          | External Data                                      |
|----------|----------------------------------------|----------------------------------------------------|
| Source   | Generated or managed within the system | Comes from outside the system                      |
| Control  | Fully controlled by the system         | Partially or fully controlled by external entities |
| Examples | User profiles, logs, settings          | API data, imported files, third-party services     |
| Risks    | Data integrity, storage limits         | Data availability, format changes, security        |

If a database is used, consider capturing the following requirements <sup>214</sup>:

- Types of information used by various functions.
- Frequency of use.

- Accessing capabilities.
- Data entities and their relationships.
- Integrity constraints.
- Data retention requirements.

Specify the following information, as required, for Internal and External Data :

- Identify the data; provide a name, if appropriate, and a brief description.
- Identify the source or destination. For external interfaces, reference the section within the IDD/ICD where the data is specified.
- Identify the units of measure required for the data, such as seconds, meters, kilohertz, etc.
- Identify the limit or range of values required for the data (for constants, provide the actual value).
- Identify the accuracy required for the data.
- Identify the precision or resolution required for the data in terms of significant digits.
- For real-time systems, identify the frequency at which the external data is calculated or refreshed, such as 10 kHz, 50 msec., etc.
- Identify legality checks performed on external data.
- Identify the data type, such as integer, American Standard Code for Information Interchange, fixed, real, enumeration, etc.
- Identify the data representation or format.

If it's better suited, this information may be captured in the project's Software Data Dictionary or Interface Control Document (ICD) and referenced in the SRS.

### 3.2.1.4 Safety Requirements

While the SRS is not required to have a specific section that addresses the safety requirements, safety requirements are to be included in the SRS and designated (marked) as safety requirements.

Software Safety Requirements define how the software must behave to prevent, detect, mitigate, or respond to hazardous conditions that could lead to injury, loss of life, environmental damage, or significant property loss. They are derived from the hazard analyses, system safety requirements, safety standards (e.g., ISO 26262, DO-178C, IEC 61508), program specification, risk assessments, vehicle or facility requirements, and interface requirements. See also [Topic 8.10 - Facility Software Safety Considerations](#).

Software Safety Requirements define the conditions, constraints, and behaviors necessary to avoid or mitigate hazards and reduce risk to an acceptable level. [SWE-134 - Safety-Critical Software Design Requirements](#) identifies the specific safety items that must be accounted for. They are:

#### SWE-134 - Safety-Critical Software Design Requirements

3.7.3 If a project has safety-critical software or mission-critical software, the project manager shall implement the following items in the software:

- a. The software is initialized, at first start and restarts, to a known safe state.
- b. The software safely transitions between all predefined known states.
- c. Termination performed by software functions is performed to a known safe state.
- d. Operator overrides of software functions require at least two independent actions by an operator.
- e. Software rejects commands received out of sequence when execution of those commands out of sequence can cause a hazard.
- f. The software detects inadvertent memory modification and recovers to a known safe state.
- g. The software performs integrity checks on inputs and outputs to/from the software system.
- h. The software performs prerequisite checks prior to the execution of safety-critical software commands.
- i. No single software event or action is allowed to initiate an identified hazard.
- j. The software responds to an off-nominal condition within the time needed to prevent a hazardous event.
- k. The software provides error handling.
- l. The software can place the system into a safe state.

The Safety Requirements should also include:

- Valid and invalid modes or states of operation (See section 3.2.1.2 above)
- All software related safety constraints between the hardware, operator, and software.
- When the software, hardware or operator performs a safety critical function, document the hardware, software, and operator roles in that function, the precedence, and the failure modes as well as any known constraints, controls, mitigations, conditions, timing constraints, limits, and wear-out factors that impact how software needs to respond.

Software Safety Requirements should be designated (marked) as safety requirements and carry a unique identification or tag for traceability purposes (see [SWE-052 - Bidirectional Traceability](#)). A way to mark and trace these requirements throughout the development and operational phases is needed to enable assessment of impacts and changes to the requirements. The unique identification or tag can be a special section in the requirements document, or a flag beside the requirement, or within a database. The method of identification is not important as long as it can be used for traceability and assessment.

Identify Software Safety Requirements to:

- Protect human life and critical assets by ensuring safe system behavior.
- Prevent or mitigate hazards caused by software faults or failures.
- Ensure compliance with safety regulations and industry standards.
- Guide design decisions that prioritize safety-critical functionality.
- Support certification and audit processes for safety-critical systems.

Software Safety Requirements are used to:

- Define safety-critical functions and their constraints.
- Specify fail-safe behaviors, redundancy, and error handling.

- Ensure traceability from hazards to software mitigations.

Types and Examples of Software Safety Requirements

| Type                             | Description                                         | Requirement Example                                                                             |
|----------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Hazard Mitigation                | Prevent or reduce the impact of identified hazards. | <i>The system shall shut down the motor if temperature exceeds 100°C.</i>                       |
| Fail-Safe Behavior               | Define safe states in case of failure.              | <i>The system shall enter a safe mode if communication with the central controller is lost.</i> |
| Redundancy and Monitoring        | Ensure backup or monitoring mechanisms.             | <i>The system shall perform health checks on redundant sensors every 5 seconds.</i>             |
| Error Detection and Handling     | Detect and respond to abnormal conditions.          | <i>The system shall log and alert on checksum errors in sensor data.</i>                        |
| Timing and Watchdog Requirements | Ensure timely execution of safety functions.        | <i>The system shall reset if the main loop fails to execute within 100 ms.</i>                  |
| Access Control and Security      | Prevent unauthorized actions that could cause harm. | <i>Only authorized users shall be able to disable safety interlocks.</i>                        |

3.2.1.5 Control System Requirements

Control system requirements specify the functional and performance expectations of a system that monitors and regulates the behavior of other systems or processes. These requirements define how the control system should sense, decide, and act to maintain desired operational conditions.

Identify control system requirements to:

- Ensure the system maintains stability, accuracy, and responsiveness under various operating conditions.
- Define how the system responds to inputs, disturbances, and faults.
- Support automation, safety, and efficiency in controlled environments.

Be sure to define and implement requirements for the following Best Practices:

- Specify and use control theory principles (e.g., PID, state-space models) to define performance metrics.
- Specify response time, accuracy, stability, and robustness.
- Identify fault detection and recovery mechanisms.
- Provide traceability to system-level functions and safety requirements.

Control system requirements are used in:

- Embedded systems (e.g., engine control units, flight controllers)
- Industrial automation (e.g., PLCs, SCADA systems)
- Robotics (e.g., motion control, path planning)
- Smart infrastructure (e.g., HVAC, lighting, traffic control)

Characteristics of Control System Requirements

| Characteristic    | Description                                                                 |
|-------------------|-----------------------------------------------------------------------------|
| Dynamic           | Involve real-time monitoring and feedback loops.                            |
| Quantifiable      | Often defined using measurable metrics (e.g., response time, error margin). |
| Time-sensitive    | Require precise timing and synchronization.                                 |
| Safety-critical   | Often tied to safety and reliability in mission-critical systems.           |
| Environment-aware | Must account for external disturbances and system variability.              |

Examples of Control System Requirements

- *The robot arm shall reach its target position within 0.5 seconds with a positional accuracy of  $\pm 0.1$  mm.*
- *The control system shall stop all motion within 200 ms of an emergency stop signal.*

3.2.1.6 Algorithm Requirements

Algorithm requirements specify the logic, rules, and computational steps that a system must follow to perform a specific task or solve a problem. They define how a function or process should be carried out, often in terms of inputs, processing logic, and expected outputs.

Identify Algorithm requirements to:

- Ensure that the intended logic is implemented correctly and consistently.
- Provide a blueprint for developers to implement complex computations or decision-making processes.
- Support traceability from high-level requirements to detailed implementation.
- Enable validation and verification of algorithm behavior.

Be sure to define and implement requirements for the following Best Practices:

- Clearly define inputs, outputs, and constraints.
- Use pseudocode, flowcharts, or state diagrams to describe logic.
- Specify timing, accuracy, and performance expectations.
- Include edge cases and exception handling.
- Provide traceability to functional and system-level requirements.

Algorithm requirements are used in:

- Control systems (e.g., PID control, trajectory planning)
- Data processing (e.g., filtering, aggregation, transformation)
- Decision-making systems (e.g., rule engines, AI/ML models)
- Safety-critical applications (e.g., collision avoidance, fault detection)

Characteristics of Algorithm Requirements

| Characteristic    | Description                                                                             |
|-------------------|-----------------------------------------------------------------------------------------|
| Deterministic     | Should produce consistent results for the same inputs (unless probabilistic by design). |
| Precise           | Must be described in unambiguous terms.                                                 |
| Testable          | Should be verifiable through simulation or testing.                                     |
| Performance-bound | Often include timing, accuracy, or resource constraints.                                |
| Traceable         | Linked to higher-level functional or control requirements.                              |
| Platform-aware    | May depend on hardware or software execution environments.                              |

Examples of Algorithm Requirements

- Control System
  - *The cruise control algorithm shall adjust throttle input every 100 ms to maintain vehicle speed within  $\pm 2$  km/h of the setpoint.*
- Signal Processing
  - *The filtering algorithm shall apply a second-order Butterworth low-pass filter with a cutoff frequency of 10 Hz to all incoming sensor data.*
- Decision Logic
  - *The system shall use a rule-based algorithm to determine whether to trigger an alert based on temperature, pressure, and vibration thresholds.*
- Machine Learning
  - *The anomaly detection algorithm shall flag any data point with a z-score greater than 3.0 as an outlier.*

3.2.1.7     Input/Output Requirements

Input/Output (I/O) requirements specify how a system receives input from users, devices, or other systems, and how it produces output in response. These requirements define the format, timing, validation, and handling of all data exchanges. I/O checks must also be performed to verify and validate the input and output data to ensure it meets expected formats, ranges, and conditions before being processed or transmitted.

Identify the I/O requirements to:

- Ensure correct and consistent data flow into and out of the system.
- Define interfaces for user interaction, hardware communication, or software integration.
- Support data validation, error handling, and feedback mechanisms.
- Ensure system reliability and safety, especially in real-time or critical systems.

Identify the I/O checking requirements to:

- Prevent invalid or unsafe data from entering the system.
- Detect and handle errors or anomalies early.
- Ensure data integrity and system stability.

Be sure to define and implement requirements for the following Best Practices:

- Define clear formats: Specify expected data types, units, and structures.
- Define data input constraints/limits: Use range checks, format checks, and type checks. Include boundary values, thresholds, and invalid inputs.
- Sanitize inputs: Prevent injection attacks or malformed data.
- Define error handling requirements: Provide requirements for meaningful error messages or fallback behaviors so that the system can fail gracefully.
- Define I/O Log events: For traceability, debugging, and auditing.

I/O requirements are used in:

- Embedded systems (e.g., sensor input, actuator output)
- User interfaces (e.g., form inputs, display outputs)
- Industrial control systems (e.g., PLC I/O)
- Software applications (e.g., file input/output, API responses)

Characteristics of I/O Requirements

| Characteristic     | Description                                                          |
|--------------------|----------------------------------------------------------------------|
| Bidirectional      | Covers both input (to the system) and output (from the system).      |
| Interface-specific | Tied to user interfaces, hardware ports, APIs, etc.                  |
| Validation-focused | Includes checks to ensure data quality and safety.                   |
| Performance-aware  | May include timing constraints (e.g., response time, sampling rate). |
| Security-sensitive | Must protect against malicious or malformed input.                   |

Examples of I/O Requirements and Checks

- Input Requirements
  - The system shall accept temperature readings from the sensor in the range of -40°C to 125°C.
  - The user shall enter a valid email address in the format `user@example.com`.
- Output Requirements
  - The system shall display a confirmation message within 2 seconds after a successful transaction.
  - The system shall send a JSON response with status code 200 upon successful API call.
- I/O Checks
  - The system shall reject any input string longer than 256 characters.
  - The system shall log and discard sensor readings that fall outside the expected range.
  - The system shall verify checksum values before processing incoming data packets.

3.2.1.8 Security and Privacy Requirements

Security and Privacy Requirements are essential to ensure the integrity and protection of mission-critical data. Security requirements define the measures and controls a system must implement to protect data, resources, services, and operations from unauthorized (accidental or malicious) access, misuse (use, modification, destruction, or disclosure), or attacks. These include mechanisms for confidentiality, integrity, availability, authentication, and authorization.

Privacy requirements define how the system must collect, store, process, and share personal or sensitive data in a way that respects user privacy and complies with legal and regulatory standards (e.g., GDPR, HIPAA, CCPA).

Identify the Security and Privacy Requirements to:

- Protect sensitive data from unauthorized access or disclosure.
- Ensure system integrity, confidentiality, and availability under normal and adverse conditions.
- Comply with industry standards and regulations.
- Safeguard user data and ensure it is handled ethically and legally.
- Prevent security breaches, data leaks, and unauthorized access.

Specific requirements in this area could include the need to:

1. "Utilize certain cryptographical techniques;
2. Keep specific log or history data sets;
3. Assign certain functions to different modules;
4. Restrict communications between some areas of the program;
5. Check data integrity for critical variables." 214

Be sure to define and implement requirements for the following Best Practices:

- Apply the principle of least privilege for access control.
- Require encryption for data at rest and in transit.
- Perform penetration testing and vulnerability scanning.
- Require audit trails and real-time monitoring.
- Ensure user consent and transparency in data collection and processing.
- Identify regulatory and industry standards to be implemented.

Security and Privacy Requirements are used in:

- System architecture and design (e.g., secure communication protocols)
- Software development (e.g., input validation, encryption)
- Compliance and auditing (e.g., data retention policies, access logs)
- Risk management (e.g., threat modeling, vulnerability assessments)

Characteristics of Security and Privacy Requirements

| Characteristic    | Description                                              |
|-------------------|----------------------------------------------------------|
| Preventive        | Designed to stop threats before they occur.              |
| Regulatory-driven | Often required by laws or standards (e.g., GDPR, HIPAA). |

|                      |                                                             |
|----------------------|-------------------------------------------------------------|
| <b>Cross-cutting</b> | Affect multiple layers of the system (UI, APIs, databases). |
| <b>Risk-based</b>    | Prioritized based on threat likelihood and impact.          |
| <b>Testable</b>      | Must be verifiable through audits, tests, or simulations.   |
| <b>Evolving</b>      | Must adapt to new threats and compliance requirements.      |

Examples of Security and Privacy Requirements

- Security Requirements
  - *The system shall require multi-factor authentication for all administrative users.*
  - *All data in transit shall be encrypted using TLS 1.3.*
  - *All user data shall be encrypted at rest and in transit.*
  - *The software shall encrypt all communication channels using AES-256.*
  - *Command sequences must be encrypted using AES-256 during transmission to prevent unauthorized control*
  - *The system shall log all access to sensitive data and retain logs for 12 months.*
  - *The application shall lock a user account after five consecutive failed login attempts.*
- Privacy Requirements
  - *The system shall allow users to view, download, and delete their personal data.*
  - *The system shall anonymize user data before sharing it with third-party analytics providers.*
  - *The system shall request explicit consent before collecting location data.*
  - *Personal data shall be stored only for the duration necessary to fulfill its intended purpose.*

3.2.1.9 Fault Management Requirements

Fault Management Requirements define how a system must detect, isolate, report, and recover from faults (errors, failures, or abnormal conditions) in both hardware and software components. These requirements ensure that the system can continue operating safely and effectively, even when faults occur. This is critical for ensuring system resilience, reliability, and safety, especially in mission-critical or high-availability environments.

Identify Fault Management requirements to:

- To minimize system downtime and maintain operational continuity.
- To prevent fault propagation that could lead to system-wide failures.
- To protect users, data, and equipment from harm or damage.
- To support diagnostics, maintenance, and recovery.
- To comply with safety and reliability standards (e.g., ISO 26262, DO-178C, IEC 61508).

Be sure to define and implement requirements for the following Best Practices:

- Perform Failure Mode and Effects Analysis (FMEA) to identify potential faults and their impacts.
- Identify redundancy requirements (e.g., dual sensors, backup modules) for critical components.
- Require use of watchdog timers to detect software hangs or crashes.
- Design for graceful degradation—allow partial functionality when full operation is not possible.
- Include self-diagnostics and health monitoring features.
- Ensure fault logging and alerting mechanisms are in place and accessible.

Fault Management requirements are used in:

- Embedded systems (e.g., avionics, automotive ECUs)
- Enterprise software (e.g., cloud services, databases)
- Industrial control systems (e.g., SCADA, robotics)
- Medical devices (e.g., infusion pumps, monitors)

Characteristics of Fault Management Requirements

| Characteristic         | Description                                                         |
|------------------------|---------------------------------------------------------------------|
| Proactive and Reactive | Includes both fault prevention and fault response mechanisms.       |
| Cross-cutting          | Applies to hardware, software, and system-level behavior.           |
| Time-sensitive         | Often includes strict timing for detection and recovery.            |
| Safety-critical        | Essential in systems where faults can lead to harm or failure.      |
| Testable               | Must be verifiable through simulation, testing, or monitoring.      |
| Traceable              | Should be linked to risk assessments and system-level requirements. |

Examples of Fault Management Requirements

- Hardware Fault Management
  - *The system shall detect power supply failures and switch to backup power within 100 ms.*

- The system shall isolate a faulty sensor and continue operation using redundant inputs.
  - The system shall log all hardware faults with timestamps and severity levels.
- Software Fault Management
  - The application shall restart the data processing module if it fails to respond within 5 seconds.
  - The system shall validate all input data and reject malformed packets to prevent buffer overflows.
  - The software shall enter a safe state if a critical exception is thrown during runtime.

3.2.1.10     Operating System Requirements

Operating System (OS) requirements specify the type, version, configuration, and capabilities of the operating system that the software or system must run on or interact with. These requirements ensure compatibility, performance, and support for necessary system-level features.

Identify OS requirements to:

- To ensure the software is compatible with the target OS environment.
- To define platform constraints for development, deployment, and testing.
- To support security, performance, and resource management needs.
- To ensure compliance with organizational or regulatory standards.

Be sure to define and implement requirements for the following Best Practices:

- Specify exact OS versions and supported platforms.
- Include minimum hardware requirements if OS performance is a factor.
- Define real-time, multitasking, or security features if needed.
- Consider long-term support (LTS) versions for stability.
- Document OS-level dependencies (e.g., services, drivers, libraries).

OS requirements are used in:

- Embedded systems (e.g., real-time OS like VxWorks, FreeRTOS)
- Enterprise applications (e.g., Windows Server, Red Hat Enterprise Linux)
- Mobile apps (e.g., Android 12+, iOS 16+)
- Cloud and containerized environments (e.g., Ubuntu 22.04 in Docker)

Characteristics of OS Requirements

| Characteristic        | Description                                                     |
|-----------------------|-----------------------------------------------------------------|
| Platform-specific     | Tied to the hardware and software environment.                  |
| Version-sensitive     | Often dependent on specific OS versions or builds.              |
| Security-relevant     | May include requirements for OS-level security features.        |
| Performance-impacting | OS choice can affect system responsiveness and resource usage.  |
| Deployment-critical   | Determines how and where the software can be installed and run. |

Examples of OS Requirements

- General Software
  - The application shall run on Windows 11 (64-bit) and macOS Ventura 13.0 or later.
  - The software shall support Ubuntu LTS versions 20.04 and 22.04.
- Embedded System
  - The control software shall run on FreeRTOS with a minimum kernel version of 10.4.3.
  - The system shall boot within 5 seconds on a Linux-based OS with real-time extensions.
- Mobile Application
  - The app shall support Android 12 and above, and iOS 16 and above.
  - The app shall use OS-level biometric APIs for authentication.

3.2.1.11     Board Support Package Requirements

A Board Support Package (BSP) is a collection of software components and configuration files that enable an operating system (typically an RTOS or embedded Linux) to run on a specific hardware platform (e.g., a microcontroller board or SoC).

These requirements define what the BSP must support, including hardware initialization, device drivers, bootloader configuration, and OS integration.

Identify BSP requirements to:

- To ensure the operating system can boot and run on the target hardware.
- To provide hardware abstraction so that application software can run independently of the underlying hardware.
- To enable early software development and testing before full hardware is available.
- To support portability and scalability across hardware variants.

Be sure to define and implement requirements for the following Best Practices:

- Require compliance with OS and hardware vendor guidelines.
- Specify testing requirements such as validate BSP functionality with hardware-in-the-loop (HIL) or emulators.

BSP requirements are used in:

- Embedded systems development (e.g., automotive ECUs, avionics, IoT devices)
- RTOS and Linux kernel porting
- Hardware bring-up and validation
- Custom board design support

Characteristics of BSP Requirements

| Characteristic    | Description                                                       |
|-------------------|-------------------------------------------------------------------|
| Hardware-specific | Tailored to a specific board or SoC.                              |
| Low-level         | Operates close to the hardware layer.                             |
| Boot-critical     | Required for system startup and OS initialization.                |
| OS-dependent      | Designed to work with a specific RTOS or Linux kernel.            |
| Reusable          | Can be adapted for similar hardware platforms.                    |
| Testable          | Must be validated through board bring-up and integration testing. |

Examples of BSP Requirements

- Hardware Initialization
  - *The BSP shall initialize the system clock, memory controller, and GPIOs during boot.*
- Device Drivers
  - *The BSP shall include drivers for UART, SPI, I2C, Ethernet, and USB interfaces.*
  - *The BSP shall support interrupt-driven UART communication.*
- Bootloader
  - *The BSP shall use U-Boot as the bootloader and support booting from NAND and SD card.*
- OS Integration
  - *The BSP shall support FreeRTOS version 10.4.3 with tickless idle mode enabled.*
  - *The BSP shall configure the Linux kernel to support device tree overlays for peripheral configuration.*
- Diagnostics and Debugging
  - *The BSP shall provide JTAG and serial console support for debugging.*
  - *The BSP shall include a memory test utility for RAM verification.*

3.2.1.12 Partitioning Requirements

Partitioning requirements define how a system’s resources, functions, and data are logically or physically separated to ensure isolation, integrity, and fault containment. Partitioning can apply to software, hardware, or both, and is often enforced by the operating system, hypervisor (aka virtual machine monitor), or hardware architecture.

Identify Partitioning requirements to:

- To prevent interference between critical and non-critical components.
- To contain faults and prevent their propagation across system boundaries.
- To enforce security and safety boundaries between different software modules or subsystems.
- To support mixed-criticality systems, where components of different assurance levels coexist.
- To enable modular development, testing, and certification.

Be sure to define and implement requirements for the following Best Practices:

- Require hardware-enforced isolation (e.g., MMU, MPU, virtualization) where possible.
- Define clear boundaries for memory, CPU time, I/O access, and privileges.
- Apply ARINC 653 or similar standards for time and space partitioning in safety-critical systems.
- Ensure partition independence: a failure in one partition must not affect others.
- Document inter-partition communication mechanisms and their constraints.

Partitioning requirements are used in:

- Real-time operating systems (RTOS) with memory and time partitioning (e.g., ARINC 653).
- Hypervisor-based systems (e.g., RedHat virtualization, Xen, QNX Hypervisor).
- Safety-critical systems (e.g., avionics, automotive ECUs).
- Multi-tenant or multi-domain systems (e.g., secure boot, sandboxing).

Characteristics of Partitioning requirements

| Characteristic    | Description                                                           |
|-------------------|-----------------------------------------------------------------------|
| Isolation-focused | Ensures that faults or attacks in one partition do not affect others. |

|                    |                                                                 |
|--------------------|-----------------------------------------------------------------|
| Deterministic      | Supports predictable behavior, especially in real-time systems. |
| Security-enhancing | Helps enforce access control and data confidentiality.          |
| Safety-critical    | Essential in systems with mixed-criticality components.         |
| Resource-bound     | Defines limits on memory, CPU, and I/O per partition.           |
| Testable           | Can be verified through simulation, testing, or formal methods. |

#### Examples of Partitioning requirements

- Software Partitioning
  - *The system shall isolate safety-critical functions from non-safety-critical functions using separate memory partitions.*
  - *Each software partition shall have a fixed time slice and shall not exceed its allocated CPU time.*
- Hardware Partitioning
  - *The system shall allocate separate memory banks for the control and diagnostic subsystems.*
  - *The system shall use a hardware memory protection unit (MPU) to enforce access control between partitions.*
- Security Partitioning
  - *The system shall separate user authentication logic from data processing logic to prevent privilege escalation.*
  - *The system shall ensure that encrypted data is processed only within a secure execution environment.*

### 3.2.1.13 Fault Tolerance and Common Mode Requirements

**Fault tolerance requirements** specify how a system must continue to operate correctly even when one or more of its components fail. These requirements define the system's ability to detect, isolate, and recover from faults without compromising safety or functionality.

**Common mode requirements** address the need to identify, prevent, or mitigate failures that occur simultaneously across multiple components or subsystems due to a shared cause (e.g., power supply failure, software bug, environmental condition).

Identify Fault Tolerance and Common Mode requirements to:

- To ensure system reliability and availability in the presence of faults.
- To prevent cascading failures and maintain safe operation.
- To identify and mitigate shared vulnerabilities that could lead to simultaneous failures.
- To support compliance with safety and reliability standards (e.g., ISO 26262, DO-178C, IEC 61508).

Be sure to define and implement requirements for the following Best Practices:

- Require redundancy (hardware, software, data paths) with independent failure modes.
- Perform Failure Mode and Effects Analysis (FMEA) and Common Cause Analysis (CCA).
- Design for graceful degradation—maintain partial functionality when full operation is not possible.
- Implement diverse redundancy (e.g., different algorithms or hardware for the same function).
- Ensure physical and logical separation of redundant components.

Fault Tolerance and Common Mode requirements are used in:

- Redundant systems (e.g., dual processors, backup sensors)
- Safety-critical applications (e.g., flight control, braking systems)
- Distributed systems (e.g., cloud infrastructure, autonomous vehicles)
- Embedded systems with real-time constraints

Characteristics of Fault Tolerance and Common Mode Requirements

| Characteristic  | Fault Tolerance                                     | Common Mode                                     |
|-----------------|-----------------------------------------------------|-------------------------------------------------|
| Focus           | Recovery from individual faults                     | Prevention of simultaneous failures             |
| Goal            | Maintain operation during faults                    | Avoid shared points of failure                  |
| Scope           | Component-level or subsystem-level                  | System-wide or cross-component                  |
| Design Strategy | Redundancy, error detection, recovery               | Diversity, isolation, independence              |
| Testability     | Verified through fault injection and stress testing | Verified through hazard and dependency analysis |
| Criticality     | Essential for high-availability systems             | Crucial for safety-critical systems.            |

#### Examples of Fault Tolerance and Common Mode Requirements

- Fault Tolerance Requirements
  - *The system shall continue operating in degraded mode if one of the redundant sensors fails.*
  - *The software shall automatically restart the communication module if it becomes unresponsive for more than 2 seconds.*
  - *The system shall detect memory corruption and switch to a backup memory bank.*
- Common Mode Requirements

- The system shall not rely on a single power source for both primary and backup controllers.
- The software and hardware watchdogs shall be implemented independently to avoid common mode failure.
- The system shall isolate redundant communication channels to prevent simultaneous failure due to EMI.

### 3.2.2 Non-Functional Requirements

Non-functional requirements (NFRs) define the quality attributes, performance constraints, and operational characteristics of a system—how the system performs its functions rather than what it does. While functional requirements describe specific behaviors or functions, non-functional requirements set the standards and constraints that shape **how** those functions are delivered.

Identify NFRs to:

- Ensure the system meets user expectations for performance, reliability, and usability.
- Define operational constraints such as response time, availability, and scalability.
- Guide architectural and design decisions.
- Support compliance with industry standards and regulations.

The SRS should focus on identifying requirements for the following types of NFRs:

- Performance and Timing (See section 3.2.2.1 for additional information)
- Software Quality Attributes (See section 3.2.2.2 for additional information)
- Design and Implementation Constraints (See section 3.2.2.3 for additional information)
- Computer Hardware Resource Utilization (See section 3.2.2.4 for additional information)

Be sure to define and implement requirements for the following Best Practices for each type of NFR:

- Use precise metrics: Make NFRs measurable and testable (e.g., use specific thresholds or metrics – Avoid vague terms like “fast” or “efficient.”).
- Define conditions: Specify under what load or scenario the requirement applies.
- Include tolerances: Use statistical thresholds (e.g., 95th percentile).
- Align with business goals: Ensure NFRs supports user expectations and mission needs.
- Document assumptions: Clarify hardware, network, and user load expectations.
- Prioritize: Identify which constraints are critical vs. desirable -- based on risk, impact, and feasibility.
- Traceable: Ensure traceability between NFRs and higher-level requirements.

NFRs are used in:

- System architecture and design (e.g., choosing frameworks, infrastructure)
- Performance tuning and optimization
- Security planning and risk management
- Testing and validation (e.g., load testing, usability testing)
- Service Level Agreements (SLAs)

Characteristics of Non-Functional Requirements

| Characteristic           | Description                                                            |
|--------------------------|------------------------------------------------------------------------|
| Quality-focused          | Concerned with how the system performs, not what it does.              |
| Cross-cutting            | Affect multiple components or the entire system.                       |
| Measurable               | Should be quantifiable (e.g., response time, uptime).                  |
| Testable                 | Must be verifiable through testing or analysis.                        |
| Constraint-driven        | Often define limits or boundaries for system behavior.                 |
| Architecture-influencing | Drive decisions about infrastructure, frameworks, and design patterns. |

Why Non-Functional Requirements Matter

- They ensure user satisfaction by defining expectations for performance, security, and usability.
- They guide architectural decisions and technology choices.
- They reduce risk by setting measurable standards for quality and reliability.

#### 3.2.2.1 Performance and Timing Requirements

Performance and timing requirements specify measurable capacities and manageable volumes of activities for the system i.e., how fast or efficiently the system operates.

 **Performance requirements** define the acceptable limits for the system's operational behavior under specific conditions. These constraints ensure the system meets expectations for speed, responsiveness, scalability, and resource usage.

 **Timing Requirements** focus specifically on time-related behavior, such as deadlines, latency, and real-time constraints.

Identify Performance and Timing Requirements to:

- Ensure system responsiveness and reliability under expected and peak loads.
- Define measurable benchmarks for system performance.
- Guide architecture and design decisions to meet performance goals (e.g., need for load balancing, caching).
- Support scalability planning for future growth.
- Prevent bottlenecks and system failures under load.
- Enable performance testing and validation.
- Ensure compliance with real-time or mission-critical timing constraints.

When capturing these requirements, define any requirements for speed, capacity, efficiency, and responsiveness. Consider including the following:

- Number of simultaneous users to be supported.
- Number of concurrent transactions and/or tasks to be supported.
- Amount of data to be processed within a specified time period (normal and peak loads).
- System response times.
- Failure recovery times.
- Output data availability.
- External hardware interface timing.
- Command execution timing.
- Maximum bandwidth throughput or capacity.

Performance and Timing Requirements give the design team a set of parameters in which to design the software solution. These requirements are used to:

- Set quantitative targets for system behavior.
- Define acceptable thresholds for time-sensitive operations (e.g., latency, throughput, and resource usage.)
- Guide design trade-offs (e.g., speed vs. accuracy).
- Establish performance baselines for testing and validation.

Examples of Performance and Timing Requirements

| Type                 | Requirement Example                                                                                                          |
|----------------------|------------------------------------------------------------------------------------------------------------------------------|
| Response Time        | <i>The system shall respond to user input within 1.5 seconds 95% of the time under normal load.</i>                          |
| Throughput           | <i>The system shall handle up to 1000 telemetry data points per second with no more than 2% packet loss under peak load.</i> |
| Latency              | <i>Data packets shall be transmitted with a maximum latency of 50 milliseconds.</i>                                          |
| Deadline             | <i>The system shall complete sensor data processing within 100 ms of receipt.</i>                                            |
| Real-Time Constraint | <i>The control system shall update actuator positions every 10 ms.</i>                                                       |
| Startup Time         | <i>The application shall initialize and be ready for use within 5 seconds.</i>                                               |
| Capacity             | <i>The system shall support 1,000 concurrent users without degradation</i>                                                   |

3.2.2.2 Software Quality Attributes

Software Quality Attributes describe how well a system performs its functions, rather than what the system does. They define the overall characteristics that affect the user experience, system performance, and maintainability.

Identify Software Quality Attributes to:

- Ensure the system meets stakeholder expectations beyond just functionality.
- Guide architectural and design decisions to support long-term system success.
- Establish measurable criteria for evaluating system performance and behavior.
- Support trade-off analysis when balancing competing priorities (e.g., performance vs. security).
- Facilitate testing and validation of system qualities.

Software Quality Attributes are used to:

- Define system-wide goals that influence architecture and design.
- Set acceptance criteria for system evaluation.
- Inform testing strategies (e.g., load testing, usability testing).

Software Quality Attributes requirements include requirements that specify software system characteristics. Define the requirements for the quality “-ilities”, as applicable:

| Attribute   | Description                                                    | Example Requirement                                                                                                                                                                           |
|-------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scalability | the ability to handle growth (e.g., users, data, transactions) | <ul style="list-style-type: none"><li>• <i>The system shall support 10,000 concurrent users.</i></li><li>• <i>The system shall scale horizontally to support increased traffic.</i></li></ul> |

|                         |                                                                                                   |                                                                                                                                                                                                                                                                                                               |
|-------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Availability</b>     | the ability to be accessed and operated when needed i.e., System uptime and readiness.            | <ul style="list-style-type: none"> <li>• <i>The system shall be available 99.9% of the time, excluding maintenance.</i></li> </ul>                                                                                                                                                                            |
| <b>Reliability</b>      | the ability to perform with correct, consistent results; ability to operate without failure       | <ul style="list-style-type: none"> <li>• <i>The system shall recover from failures within 30 seconds.</i></li> <li>• <i>The system shall have a mean time between failures (MTBF) of 1,000 hours.</i></li> <li>• <i>The system shall sustain 99.99% uptime during mission-critical operations.</i></li> </ul> |
| <b>Usability</b>        | the ability to be easily learned and used                                                         | <ul style="list-style-type: none"> <li>• <i>New users shall complete onboarding within 5 minutes.</i></li> <li>• <i>The interface shall comply with WCAG 2.1 accessibility standards.</i></li> </ul>                                                                                                          |
| <b>Maintainability</b>  | the ability to be easily updated or fixed                                                         | <ul style="list-style-type: none"> <li>• <i>Code changes shall not require more than 2 hours of regression testing.</i></li> <li>• <i>Code shall be modular and documented to support future enhancements.</i></li> </ul>                                                                                     |
| <b>Portability</b>      | the ability to be easily modified/adapted to run in a new environment or on a different platform. | <ul style="list-style-type: none"> <li>• <i>The application shall run on Windows, macOS, and Linux.</i></li> </ul>                                                                                                                                                                                            |
| <b>Interoperability</b> | the ability to work with other systems.                                                           | <ul style="list-style-type: none"> <li>• <i>The application shall integrate with external APIs using REST.</i></li> </ul>                                                                                                                                                                                     |
| <b>Testability</b>      | the ability to be easily and thoroughly tested; Ease of verifying system behavior.                | <ul style="list-style-type: none"> <li>• <i>All modules shall have at least 80% unit test coverage.</i></li> </ul>                                                                                                                                                                                            |
| <b>Reusability</b>      | the ability to be used in multiple applications                                                   | <ul style="list-style-type: none"> <li>• <i>The XYZ API shall provide timing services to applications on the system.</i></li> </ul>                                                                                                                                                                           |

See also [8.02 - Software Quality](#)

### 3.2.2.3 Design and Implementation Constraints

Design and Implementation Constraints are limitations or restrictions that affect how a software system is designed, developed, or implemented. These non-functional requirements must be adhered to during the software development lifecycle. They address constraints that "can be imposed by other standards, hardware limitations, etc." <sup>214</sup> Document any constraints imposed by the system, hardware, standards, compliance, regulations, or tools.

Identify Design and Implementation Constraints to:

- Ensure compliance with organizational, technical, or legal standards.
- Guide developers in making architectural and design decisions.
- Prevent rework by clarifying limitations early in the project.
- Align expectations among stakeholders, developers, and testers.

Design and Implementation Constraints are used to:

- Set boundaries for design choices.
- Inform architecture and technology stack decisions.
- Highlight mandatory tools, platforms, or frameworks.
- Identify integration or interoperability requirements.

Examples of Design and Implementation Constraints

Here are some common examples of design and implementation constraints:

| Type                 | Example of Constraints                   |
|----------------------|------------------------------------------|
| Hardware             | Must run on Raspberry Pi 4 with 4GB RAM. |
| Software             | Must use PostgreSQL as the database.     |
| Programming Language | Must be implemented in Java.             |
| Framework            | Must use React for the frontend.         |
| Security             | Must comply with HIPAA regulations.      |
| Performance          | Must support 1000 concurrent users.      |
| Integration          | Must integrate with existing SAP system. |

|            |                                                                            |
|------------|----------------------------------------------------------------------------|
| Deployment | Must be containerized using Docker.                                        |
| Safety     | Must comply with the MISRA C coding standards for safety-critical systems. |

Additional "examples include requirements concerning

- a. Use of a particular [Computer Software Configuration Item] CSCI architecture or requirements on the architecture, such as required databases or other software units, use of standard, military, or existing components, or use of Government- or acquirer-furnished property (equipment, information, or software)
- b. Use of particular design or implementation standards, use of particular data standards, and use of a particular programming language
- c. Flexibility and expandability that must be provided to support anticipated areas of growth or changes in technology, threat, or mission." [214](#)

3.2.2.4 Computer Hardware Resource Utilization Requirements

Computer Hardware Resource Requirements specify the minimum and recommended hardware specifications needed to run the software effectively. This includes CPU, memory (RAM), storage, GPU, network bandwidth, and other peripheral or specialized hardware.

Utilization Requirements define how efficiently the software should use these resources under various operating conditions, including normal, peak, and idle states. Computer hardware resource utilization includes items such as "maximum allowable use of processor capacity, memory capacity, input/output device capacity, auxiliary storage device capacity, and communications or network equipment capacity. The requirements (e.g., stated as percentages of the capacity of each computer hardware resource) include the conditions, if any, under which the resource utilization is to be measured."

Identify hardware and utilization requirements to:

- Ensure compatibility with target deployment environments.
- Prevent performance degradation due to insufficient resources.
- Support capacity planning and infrastructure provisioning.
- Ensure system communication capabilities for data transmission
- Guide testing and benchmarking efforts.

Be sure to define memory, processor, storage, and other computer resource requirements including communication requirements. Communications requirements may include items such as "geographic locations to be linked, configuration and network topology, transmission technique, data transfer rates, gateways, required system use times, type and volume of data to be transmitted or received, time boundaries for transmission, reception, and response, peak volumes of data, and diagnostic features."

These requirements are used to:

- Define baseline hardware specifications for development, testing, and production.
- Set resource usage thresholds for performance monitoring.
- Inform deployment decisions (e.g., cloud vs. on-premises, edge devices).

Examples of Computer Hardware Resource Requirements

| Type        | Requirement Example                                                                      |
|-------------|------------------------------------------------------------------------------------------|
| CPU         | <i>The system shall require a minimum of 4-core, 2.5 GHz processor.</i>                  |
| Memory      | <i>The application shall operate with at least 8 GB of RAM.</i>                          |
| Storage     | <i>The system shall require 20 GB of free disk space for installation and operation.</i> |
| GPU         | <i>The software shall support NVIDIA CUDA-enabled GPUs for AI processing.</i>            |
| Network     | <i>The system shall require a minimum of 100 Mbps network bandwidth.</i>                 |
| Utilization | <i>CPU usage shall not exceed 75% under peak load for more than 10 minutes.</i>          |
| Power       | <i>The embedded system shall operate within a 10W power envelope.</i>                    |

3.2.3 Interface Requirements

Interface requirements define how the software interacts with other systems, hardware, users, or other software components. These are critical for integration and interoperability as they define the boundaries and communication protocols between the system and its environment.

Interface requirements cover all inputs and outputs for the software system to pass data to its appropriate location. There are a multitude of interface types that software systems may need to utilize to fulfill the software's requirements. Some of the interfaces the software may have include external interfaces, internal interfaces, user interfaces, system interfaces, hardware interfaces, communication interfaces, and interfaces with services.

Identify Interface requirements to:

- Ensure seamless integration with external systems or components.
- Define communication protocols and data exchange formats.
- Prevent compatibility issues during development and deployment.
- Support modularity and system scalability.
- Enable testing and validation of system interactions.

To effectively define and manage end Interface requirements, be sure to perform the following Best Practices:

- **Use Interface Definition Languages (IDLs):** For structured APIs (e.g., gRPC, Thrift, OpenAPI).
- **Use diagrams:** Include interface diagrams (e.g., context diagrams, sequence diagrams, interface mockups) to visualize interactions.
- **Specify data formats:** Clearly define input/output formats (e.g., JSON schema, XML structure).
- **Include error handling:** Describe how the system should respond to invalid inputs or failed connections.
- **Document dependencies:** Note any third-party systems, APIs, or hardware the interface depends on.
- **Version control:** Document versions of APIs, protocols, and interfaces to manage compatibility.
- **Use consistent terminology:** Align with industry standards and naming conventions and data types.
- **Validate with stakeholders:** Confirm interface expectations with developers, integrators, and end-users.

Interface requirements are used to:

- Describe how the system connects to external systems or devices.
- Specify data formats, message structures, and protocols, reducing ambiguity.
- Define input/output behavior for each interface.
- Serve as a reference for developers and integrators.

Note that interface specifications may be captured in a separate interface requirements document; in that case, a reference to the separate document needs to be included in the SRS.

See also Topic [5.02 - IDD - Interface Design Description](#).

### 3.2.3.1 External Interface Requirements

External Interface Requirements define how the software system interacts with external entities such as:

- Other software systems
- Hardware devices
- Users (via user interfaces)
- Communication networks
- Databases or storage systems
- Other external entities such as sensors and ground stations

External Interface Requirements cover all inputs and outputs for the software system and expand on the interfaces. These requirements specify the data formats, protocols, communication methods, and interaction rules necessary for successful integration and interoperability. When capturing external interface requirements, consider including interfaces to items such as test equipment or transportation systems, CSCI-to-hardware, CSCI-to-CSCI, and/or CSCI-to-external facilities.

Each external interface is identified by name and source. Any identifying documentation, such as an ICD, is referenced for each interface. An external interface diagram may be used as a description aid.

Information such as the following is included, as applicable to the CSCI/system:

- Identifier (name).
- Description.
- Source (input) or destination (output).
- Valid range. Include Error handling and response codes.
- Units of measure.
- Timing.
- Data formats. (e.g., JSON, XML)
- Command formats.
- Type of interface, e.g., real-time data transfer, storage-and-retrieval of data.
- Authentication methods (e.g., OAuth2)
- Characteristics, e.g., data type, size, format, security, frequency, of data elements that the software must provide, store, send, access, receive.
- Characteristics of communication methods, e.g., message format, communication bands, transfer rate, API endpoints and protocols (e.g., REST, SOAP).
- Dependencies, versions, and Service Level Agreements (SLAs).

Identify External Interface Requirements to:

- Ensure compatibility with external systems and components.
- Facilitate integration with external applications, APIs, platforms third-party services, hardware, or legacy systems.
- Define clear boundaries between the system and its environment.
- Support testing and validation of system interactions.
- Prevent miscommunication between development teams and external stakeholders.

External Interface Requirements are used to:

- Describe how the system communicates with external components or entities.
- Specify input/output formats, data structures, and protocols.
- Define authentication, error handling, and data validation rules.

Examples of External Interface Requirements

- *The system shall integrate with the PayPal API to process online payments using REST over HTTPS.*
- *The Power Management Software must interface with onboard solar panel controllers to report real-time power generation to the telemetry subsystem.*
- *All external communication shall use HTTPS with TLS 1.3 encryption.*

See also Topic [5.02 - IDD - Interface Design Description](#).

3.2.3.2 Internal Interface Requirements

Internal Interface Requirements define how components, functions, modules, or subsystems within the same software system interact with each other. These interfaces are not exposed to external systems or users but are critical for ensuring modularity, maintainability, and internal consistency. When capturing internal interface requirements, information such as that noted above for **External Interface Requirements** applies, as appropriate. Data flow diagrams may be used to illustrate the interfaces if they are not left to the design phase. Note that software design interface specifications are captured in an [5.02 - IDD - Interface Design Description](#), which needs to be referenced in the SRS.

Identify Internal Interface Requirements to:

- Ensure clear communication and data exchange between internal components including boundaries and communication protocols.
- Support modular design and separation of concerns.
- Facilitate parallel development by different teams or developers.
- Enable easier testing and debugging of individual modules.
- Promote maintainability and scalability of the system architecture.

Internal Interface Requirements are used to:

- Define data exchange formats, function signatures, and communication protocols between internal modules.
- Specify dependencies, timing, and error handling mechanisms.
- Serve as a contract between development teams working on different parts of the system.

Types of Internal Interfaces

| Type                       | Description                                       | Examples                                                              |
|----------------------------|---------------------------------------------------|-----------------------------------------------------------------------|
| Module-to-Module Interface | Defines how two software modules interact.        | Function calls, shared memory, message queues.                        |
| Service Interface          | Defines how internal services communicate.        | Internal REST endpoints, gRPC, service buses.                         |
| Data Interface             | Specifies how data is passed between components.  | Data structures, serialization formats (e.g., JSON, Protobuf).        |
| API Interface              | Internal APIs used by other parts of the system.  | Internal SDKs, utility libraries.                                     |
| Layered Interface          | Defines interaction between architectural layers. | UI layer calling business logic, business logic accessing data layer. |

Examples of Internal Interface Requirements

- *The authentication module shall expose a validateUser(token) method to the session manager.*
- *The telemetry subsystem must receive attitude data from XYZ and pass it to the ground communication subsystem for transmission every 10 milliseconds.*
- *The TelemetryModule shall send sensor data to the DataLogger in JSON format with the following schema: { "timestamp": ISO8601, "sensorId": string, "value": float }.*
- *The NotificationService shall receive messages from the OrderService via an internal message queue using RabbitMQ.*

3.2.3.3 User Interface Requirements

User Interface Requirements define how users interact with the software system, typically through graphical or command-line interfaces. These requirements describe the look, feel, behavior, and accessibility of the interface, including layout, navigation, input methods, visual elements, and responsiveness. They ensure the system is usable, intuitive, and accessible to its intended users.

User interface requirements between the user and the software are commonly called Human-Computer Interface (HCI) requirements. These interfaces can be physical, visual, auditory, or cognitive, and they play a crucial role in usability, accessibility, and user experience. When writing these requirements, include the mode of interaction (e.g., command line, graphical user interface, input script) and requirements for specific capabilities (e.g., command line options, screen formats, batch processing.) To derive the user interface requirements, the software developer may need to develop a prototype and iterate with end-user and customer to dial in on the final set of requirements.

Identify User Interface Requirements to:

- Ensure usability and accessibility for all user types.
- Guide UI/UX design and development with clear expectations.
- Align the interface with user needs and workflows.
- Support consistency across screens, platforms, and devices.
- Facilitate testing of visual and interactive elements.

When gathering the user interface requirements, the following guidelines should also be considered: <sup>697</sup>

1. **Visibility of system status:** Users need to know what is happening throughout their interaction with the product.
2. **Match between system and the real world:** Design elements should be familiar and intuitive.
3. **User control and freedom:** Users should have the ability to undo actions and navigate freely.
4. **Consistency and standards:** Follow established design patterns and conventions.
5. **Error prevention:** Design to minimize user errors.
6. **Recognition rather than recall:** Avoid requiring users to remember complex information.
7. **Flexibility and efficiency of use:** Accommodate both novice and expert users.
8. **Aesthetic and minimalist design:** Keep the interface clean and visually appealing.

### 9. Help users recognize, diagnose and recover from errors

### 10. Help and documentation: Provide concise, searchable documentation.

Be sure to specify the logical characteristics of each interface between the software product and its users.

 **Note:** A style guide for the user interface can provide consistent rules for organization, coding and interaction of the user with the system. [696](#)

User Interface Requirements are used to:

- Define how users interact with the system.
- Specify UI elements, layouts, and navigation flows.
- Describe input/output behavior, error messages, and feedback mechanisms.
- Ensure compliance with accessibility standards (e.g., Web Content Accessibility Guidelines (WCAG)).
- Define supported devices (e.g., web, mobile, or desktop applications)

Common Types of User Interfaces include:

1. **Command-Line Interface (CLI)** – Text-based interface where users type commands and the computer responds by outputting text to the monitor.
  - Example: Terminal, PowerShell, Bash.
  - Use Case: System administration, programming, automation.
  - Pros: Precise, powerful, scriptable.
  - Cons: Steep learning curve, not user-friendly for beginners.
2. **Graphical User Interface (GUI)** – Visual interface with windows, icons, buttons, and menus. Accepts input via devices such as a computer keyboard and mouse.
  - Use Case: General-purpose computing, desktop and mobile apps.
  - Pros: Intuitive, user-friendly, visually rich.
  - Cons: Resource-intensive, less efficient for repetitive tasks.
3. **Touch User Interface (TUI)** – GUIs that responds to touch gestures via a [touchpad](#) or touchscreen.
  - Example: Smartphones, tablets, touchscreens in kiosks.
  - Use Case: Mobile apps, point-of-sale systems, ATMs.
  - Pros: Natural interaction, portable.
  - Cons: Limited precision, not ideal for complex tasks.
4. **Voice User Interface (VUI)** – Interaction through spoken commands. Input is made by pressing keys or buttons or responding verbally to the interface.
  - Example: Siri, Alexa, Google Assistant.
  - Use Case: Smart homes, accessibility tools, hands-free environments.
  - Pros: Hands-free, accessible.
  - Cons: Limited by speech recognition accuracy and context understanding.
5. **Natural Language Interface (NLI)** – Allows users to interact using natural language (typed or spoken). Users type in a question and wait for a response.
  - Example: Chatbots, virtual assistants, AI systems like Copilot.
  - Use Case: Customer service, information retrieval, productivity tools.
  - Pros: Conversational, intuitive.
  - Cons: Ambiguity in language can lead to misunderstandings.
6. **Gesture-Based Interface** – Uses body movements or hand gestures to control the system.
  - Example: Microsoft Kinect, Leap Motion, VR hand tracking.
  - Use Case: Gaming, virtual reality, accessibility.
  - Pros: Immersive, hands-free.
  - Cons: Requires sensors, can be tiring or imprecise.
7. **Batch Interface** – Non-interactive user interface. User specifies all the details in advance and receives the output when all batch processing is done.
  - Example: Shell scripts, simulations.
  - Use Case: System administration, programming, automation.
  - Pros: Precise, powerful, process large amounts of data in short amount of time.
  - Cons: Steep learning curve, not user-friendly for beginners.

Examples of User Interface Requirements

- *The login screen shall include fields for username and password, and a "Forgot Password" link.*
- *The UI shall collapse the sidebar into a hamburger menu on screens narrower than 768px.*
- *All images shall include descriptive alt text for screen readers.*
- *The homepage shall display a welcome message, a search bar, and a list of recent activities in a vertical layout.*
- *Widget XYZ shall have the following three options: Red, Yellow, Green.*
- *The system shall support dark mode and high-contrast themes.*
- *The system shall support inputs from a shell script written in Bash.*

3.2.3.4 System Interface Requirements

System Interface Requirements define how the software system interacts with other systems, libraries, or APIs —which may be internal subsystems or external systems. These interfaces specify the data exchange, communication protocols, integration points, and interaction rules between the system under development and other systems within the same environment.

List each system interface and identify the functionality of the software to accomplish the system requirement and the interface description to match the system. 696

Identify System Interface Requirements to:

- Ensure interoperability between systems.
- Define integration boundaries and responsibilities.
- Facilitate data exchange and synchronization.
- Prevent integration issues during development and deployment.
- Support system scalability and modularity.

Be sure to Include:

- Data exchange mechanisms including data formats and protocols.
- Timing/scheduling of interactions.
- Security, audit, and access control requirements.

System Interface Requirements are used to:

- Describe how the system communicates with other systems.
- Specify data formats, protocols, and authentication mechanisms.
- Define input/output behavior, timing, and error handling.

Types of System Interface Requirements

| Type                                    | Description                                               | Examples                                           |
|-----------------------------------------|-----------------------------------------------------------|----------------------------------------------------|
| Application Programming Interface (API) | Interfaces for software-to-software communication.        | REST, SOAP, GraphQL APIs.                          |
| Database Interface                      | Interfaces for accessing or sharing data between systems. | Shared databases, data replication, ETL pipelines. |
| File-Based Interface                    | Systems exchange data via files.                          | CSV, XML, JSON files over FTP/SFTP.                |
| Message-Based Interface                 | Asynchronous communication via messaging systems.         | Kafka, RabbitMQ, MQTT.                             |
| Web Services Interface                  | Web-based communication using standard protocols.         | HTTP/HTTPS with JSON/XML payloads.                 |
| Middleware Interface                    | Communication through integration platforms.              | ESB, iPaaS, service buses.                         |

Examples of System Interface Requirements

| Type                    | Requirement Example                                                                                                                                                                       |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API Interface           | <i>The system shall expose a REST API at /api/v1/inventory that accepts JSON-formatted POST requests and returns HTTP status codes.</i>                                                   |
| Database Interface      | <i>The system shall read customer data from a shared PostgreSQL database using read-only credentials.</i>                                                                                 |
| File-Based Interface    | <i>The system shall export daily transaction logs in CSV format to an SFTP server by 2:00 AM UTC.</i>                                                                                     |
| Message-Based Interface | <i>The system shall publish order status updates to the order_updates topic on Apache Kafka.</i>                                                                                          |
| Web Services Interface  | <i>The system shall consume weather data from the external service at <a href="https://api.weather.com/v3/forecast">https://api.weather.com/v3/forecast</a> using HTTPS and API keys.</i> |

3.2.3.5 Hardware Interfaces

Hardware Interface Requirements specify how the software system interacts with physical hardware components. These requirements define the electrical, mechanical, and data communication protocols used to connect and communicate with hardware devices. They ensure that the software can correctly control, monitor, or receive data from hardware elements such as sensors, actuators, controllers, or embedded systems.

Specify the logical characteristics of each interface between the software product and the hardware elements of the system. This includes configuration characteristics (number of ports, instruction sets, etc.). It also covers such matters as what devices are to be supported, how they are to be supported, and protocols. For example, terminal support may specify full-screen support as opposed to line-by-line support. 696

Identify Hardware Interface Requirements to:

- Ensure compatibility between software and hardware components.
- Define communication protocols (e.g., USB, Bluetooth) and data formats for hardware interaction.
- Support integration and testing of hardware-software systems.
- Prevent hardware-related failures due to miscommunication or mismatched specifications.
- Enable maintainability and scalability of hardware-dependent systems.

Hardware Interface Requirements are used to:

- Describe how the software communicates with hardware devices.
- Specify data exchange formats, timing, voltage levels, and protocols.
- Define input/output behavior, error handling, and device control logic.

Types of Hardware Interface Requirements

| Type                   | Description                                            | Examples                               |
|------------------------|--------------------------------------------------------|----------------------------------------|
| Digital I/O            | Binary signals for control or status.                  | GPIO pins, relay control.              |
| Analog I/O             | Continuous signals for sensors or actuators.           | Temperature sensors, potentiometers.   |
| Serial Communication   | Byte-stream communication.                             | UART, RS-232, RS-485.                  |
| Parallel Communication | Multiple data lines for simultaneous transmission.     | Data buses in embedded systems.        |
| Network Interfaces     | Communication over Ethernet or wireless.               | TCP/IP, CAN bus, Modbus TCP.           |
| USB Interfaces         | Universal serial communication.                        | USB barcode scanners, USB HID devices. |
| I2C/SPI Interfaces     | Synchronous serial communication for embedded systems. | EEPROM, accelerometers, displays.      |

Examples of Hardware Interface Requirements

| Type                 | Requirement Example                                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------------------------------|
| Digital I/O          | <i>The software shall set GPIO pin 17 to HIGH to activate the cooling fan when the temperature exceeds 75°C.</i>    |
| Serial Communication | <i>The system shall communicate with the GPS module over UART at 9600 baud, 8 data bits, no parity, 1 stop bit.</i> |
| USB Interface        | <i>The software shall detect and read input from a USB-connected barcode scanner using the HID protocol.</i>        |
| I2C Interface        | <i>The system shall read temperature data from the I2C sensor at address 0x48 every 500 ms.</i>                     |
| CAN Bus              | <i>The software shall transmit vehicle speed data over the CAN bus using message ID 0x0C0.</i>                      |

3.2.3.6    Communication interfaces

Communication Interface Requirements describe how the software system exchanges data with other systems, components, or devices over a network or communication channel. These requirements define the protocols, data formats, timing, security, and error handling mechanisms used in communication. They are essential for ensuring interoperability, data integrity, and secure transmission between distributed systems.

If not already covered in other areas, specify the various interfaces to communications. Define protocols, ports, and encryption standards including timeout, retry, and failover strategies. Also document any firewall and network configuration requirements.

Identify Communication Interface Requirements to:

- Ensure reliable and secure data exchange between systems.
- Define communication protocols (e.g., HTTPS, MQTT, WebSocket) and standards to be used.
- Facilitate integration with external systems, services, or devices.
- Support scalability and performance in distributed environments.
- Prevent communication failures due to mismatched expectations.

Communication Interface Requirements are used in client-server, distributed, or cloud-based systems to:

- Specify how data is transmitted (e.g., protocol, port, frequency).
- Define message formats, timing, and synchronization.
- Describe security mechanisms (e.g., encryption, authentication).
- Outline error detection and recovery strategies.

Types of Communication Interface Requirements

| Type                          | Description                             | Examples                     |
|-------------------------------|-----------------------------------------|------------------------------|
| Network Communication         | Data exchange over LAN/WAN/Internet.    | TCP/IP, UDP, HTTP, HTTPS.    |
| Message-Based Communication   | Asynchronous messaging between systems. | MQTT, AMQP, Kafka, RabbitMQ. |
| Remote Procedure Call (RPC)   | Invoking functions on remote systems.   | gRPC, XML-RPC, JSON-RPC.     |
| Web Services                  | Communication via web-based APIs.       | REST, SOAP, GraphQL.         |
| Fieldbus/Industrial Protocols | Used in embedded or industrial systems. | CAN, Modbus, Profibus.       |

|                        |                                        |                                 |
|------------------------|----------------------------------------|---------------------------------|
| Wireless Communication | Data exchange over wireless protocols. | Bluetooth, Wi-Fi, Zigbee, LoRa. |
|------------------------|----------------------------------------|---------------------------------|

#### Examples of Communication Interface Requirements

| Type                        | Requirement Example                                                                                                                                          |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Network Communication       | <i>The system shall communicate with the central server over TCP/IP on port 443 using HTTPS.</i>                                                             |
| Message-Based Communication | <i>The application shall publish telemetry data to the sensor_data topic on MQTT broker every 5 seconds.</i>                                                 |
| Web Services                | <i>The system shall consume weather data from <a href="https://api.weather.com/v3/forecast">https://api.weather.com/v3/forecast</a> using REST and JSON.</i> |
| RPC                         | <i>The client shall invoke the getUserProfile(userId) method on the remote service using gRPC.</i>                                                           |
| Industrial Protocol         | <i>The system shall read temperature values from a Modbus RTU slave device at address 0x01 every 2 seconds.</i>                                              |

### 3.2.3.7 Interfaces with Services

Service Interface Requirements define how a software system interacts with external services, which may include: cloud-based services (e.g., SaaS, PaaS), third-party APIs, enterprise systems, microservices or internal services. These requirements specify the communication protocols, data formats, authentication mechanisms, and interaction rules necessary for successful integration.

Identify Service Interface Requirements to:

- Ensure reliable and secure integration with external services.
- Define technical expectations for data exchange and communication including retry and failover mechanisms
- Support maintainability and scalability of service-oriented architectures.
- Prevent integration issues due to mismatched protocols or data formats.
- Enable compliance with service-level agreements (SLAs), security, and regulatory standards.

Service Interface Requirements are used to:

- Describe how the system connects to and interacts with services.
- Specify service endpoints, protocols, and data formats.
- Define authentication, rate/usage limits, and error handling.
- Document dependencies and versioning of external services.

#### Types of Service Interface Requirements

| Type                    | Description                                          | Examples                         |
|-------------------------|------------------------------------------------------|----------------------------------|
| Web APIs                | REST, SOAP, or GraphQL interfaces for data exchange. | Google Maps API, OpenWeather API |
| Authentication Services | Identity and access management.                      | Auth0, Okta, Azure AD            |
| Payment Services        | Transaction processing and billing.                  | Stripe, PayPal, Square           |
| Notification Services   | Email, SMS, or push notifications.                   | Twilio, SendGrid, Firebase       |
| Data Services           | Access to external datasets or analytics.            | Google Analytics, AWS Athena     |
| Cloud Storage Services  | File upload/download and storage.                    | Dropbox, Google Drive, Amazon S3 |
| Enterprise Services     | Integration with ERP, CRM, or HR systems.            | Salesforce, SAP, Workday         |

#### Examples of Service Interface Requirements

| Type                   | Requirement Example                                                                                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| REST API Integration   | <i>The system shall retrieve weather data from <a href="https://api.weather.com/v3/forecast">https://api.weather.com/v3/forecast</a> using HTTPS and JSON format.</i> |
| Authentication Service | <i>The system shall authenticate users via OAuth 2.0 using Auth0, with access tokens valid for 60 minutes.</i>                                                        |
| Payment Gateway        | <i>The system shall process payments through Stripe using the /v1/charges endpoint with bearer token authentication.</i>                                              |
| Email Notification     | <i>The system shall send password reset emails via SendGrid using the /v3/mail/send endpoint.</i>                                                                     |

## 3.2.4 User Requirements

User Requirements describe the needs, expectations, and interactions of the people who will directly use the software system. These requirements focus on usability, accessibility, functionality, and user experience from the perspective of the end user. They describe what the end user needs the system to do, often in natural language or user stories and focus on the goals, tasks, and experiences of the user, rather than the technical implementation. Define user-specific needs, focusing on usability, personalization, and human-machine interaction.

Identify User Requirements to:

- Ensure the software meets user expectations and supports their goals.
- Improve usability and satisfaction by aligning features with real-world tasks. This helps ensure the system is usable, relevant, and valuable to its intended users.
- Guide UI/UX design and workflow development.
- Support training, onboarding, and documentation efforts.
- Reduce rework by validating user needs early in the development process.
- Serve as a bridge between business requirements and functional/system requirements.

To effectively define and manage end user requirements, be sure to perform the following Best Practices:

- Engage real users: Conduct interviews, surveys, or usability studies to gather input.
- Use personas and scenarios: Describe typical users and how they interact with the system.
- Be clear and specific: Avoid vague terms like “easy to use”; define what that means.
- Prioritize requirements: Identify which features are essential vs. nice-to-have.
- Include acceptance criteria: Define how each requirement will be validated.
- Iterate with feedback: Refine requirements based on user testing and stakeholder input.
  - Validate them through prototypes, mockups, or user testing.

User Requirements are used to:

- Define what the user can do with the system.
- Describe how the system should behave in response to user actions.
- Inform interface design, accessibility features, and user workflows.

Characteristics of User Requirements

- User-centered: Focused on the needs, tasks, and goals of the end user. Help define the scope of the system from a user’s point of view.
- Task-oriented: Describe what users need to accomplish.
- Observable: Can be validated through user testing or feedback.
- Context-aware: Consider the environment in which the software is used.
- Inclusive: Address accessibility, language, and skill level differences.
- Often expressed as user stories, use cases, or scenarios.

Relationship to Other Requirements

- Business Requirement: Improve customer engagement.
- User Requirement: As a user, I want to receive notifications about new features.
- Functional Requirement: The system shall send an email notification to users when a new feature is released.

Examples of User Requirements

| Type           | Requirement Example                                                           |
|----------------|-------------------------------------------------------------------------------|
| Functional     | <i>The user shall be able to reset their password via email verification.</i> |
| Usability      | <i>The system shall provide tooltips for all form fields.</i>                 |
| Accessibility  | <i>The application shall comply with WCAG 2.1 AA standards.</i>               |
| Localization   | <i>The user interface shall support English and Spanish languages.</i>        |
| Customization  | <i>Users shall be able to configure dashboard widgets.</i>                    |
| Help & Support | <i>The system shall provide in-app help and a searchable knowledge base.</i>  |

- Agile Examples:
  - User Stories (Agile Format)
    - As a user, I want to view my order history so I can track past purchases.
    - As a customer, I want to track my order status so that I know when it will arrive.
    - As an admin, I want to manage user accounts so that I can control access to the system.
  - Use Case Format
    - Title: Submit Feedback
    - Actor: Registered User
    - Description: The user submits feedback through a form, which is stored in the database and emailed to the support team.

### 3.3 Qualification Provisions

Qualification Provisions are the planned methods, criteria, and conditions used to demonstrate that a software system or component meets its specified requirements. These provisions define the method for how each requirement will be verified or validated, typically through testing, analysis, inspection, or demonstration.

So, in addition to the requirements themselves, their verification methods should be included in the SRS, either when the requirement is stated or in a separate verification matrix (if in a separate document, provide a reference.) For each requirement, clearly outline the methodology for verifying and validating requirements to ensure traceability and compliance. Maintain an SRS traceability matrix mapping requirements to tests (see [SWE-052 - Bidirectional Traceability](#)).

Types of Qualification Methods

| Method        | Description                                     | When Used                                                                                    |
|---------------|-------------------------------------------------|----------------------------------------------------------------------------------------------|
| Test          | Execute the system to observe behavior.         | Functional including safety, real-time performance, scalability, and interface requirements. |
| Analysis      | Use models, simulations, or calculations.       | Performance, timing, or algorithmic requirements.                                            |
| Inspection    | Review documentation, code, or artifacts.       | Design, documentation, or coding standards.                                                  |
| Demonstration | Show functionality in a controlled environment. | User interface, operational workflows, complex integration or edge cases.                    |

Examples of Qualification Provisions

| ID    | Requirement Example                                                           | Qualification Rationale                                                | Qualification Method |
|-------|-------------------------------------------------------------------------------|------------------------------------------------------------------------|----------------------|
| REQ-1 | <i>The system shall encrypt all data at rest using AES-256.</i>               | Inspection of configuration and analysis of encryption implementation. | Inspection           |
| REQ-2 | <i>The system shall respond to user input within 2 seconds.</i>               | Test under simulated load conditions.                                  | Test                 |
| REQ-3 | <i>The system shall display a confirmation message after form submission.</i> | Demonstration during UI walkthrough.                                   | Demonstration        |
| REQ-4 | <i>The software shall comply with coding standard XYZ.</i>                    | Inspection of source code and static analysis reports.                 | Inspection, Analysis |

See also Topic [5.14 - Test - Software Test Procedures](#).

3.4 Rationale and Supporting Information

Supporting rationale provides additional information to help clarify the intent of the requirements at the time they were written. The rationale should explain/justify why each requirement exists and its importance to mission success. The supporting rationale may be located with the requirements to which it applies. Consider including the following in the requirements rationale to provide context and justification:

- Reason for the requirement.
- Assumptions, e.g., requirement assumes availability of new technology development.
- Relationships, e.g., requirement is based on expectation for how users will interact with the software.
- Design constraints, e.g., requirements written based on trade studies or decisions made during the design phase.
- Supporting information referenced in external documentation.

Guidance for [SWE-050 - Software Requirements](#) also provides information on requirements rationale.

Example:

- Rationale: The real-time pressure monitoring requirement for propulsion systems is essential to prevent potential failures during orbital insertion due to fluctuating fuel chamber conditions.

3.5 Additional Requirements and Information

In order to build the software, there may be other requirements that must be complied with or information that is instrumental to the success of the project. They should be captured and documented also. They may include:

1. Legal, regulatory, or compliance requirements.
2. Use cases, scenarios, or workflows illustrating the requirements.
3. Glossary of terms or abbreviations for clarity.
4. References to related documents, policies, standards, or specifications.
5. End of Life/Retirement/Decommissioning requirements.
6. Supporting information (see IEEE 29148 section 9.6.20)

3.6 Assumptions and Limitations

It is best to obtain definitive answers from the user community and software/system designers before making any assumptions. In the event clarification cannot be provided, document all assumptions made, along with any limitations, related to hardware behavior, operator inputs, environmental conditions, or mission-critical scenarios. Provide context for each and allow for validation during system design and testing.

- Hardware assumptions: Clearly state expectations around hardware (e.g., sensor accuracy, actuation speed, latency), ensuring software requirements account for system limitations.
- Operator assumptions: Define assumptions about operator behavior, skills, training levels, and familiarity with the software systems.
- Environmental assumptions: Reflect assumptions on environmental conditions (e.g., radiation levels, atmospheric conditions, gravitational forces).

Examples:

- It is assumed that temperature sensors will provide accurate readings within ±2°C.
- It is assumed that operators are trained to manually override automated controls in critical situations.
- Software is developed under the assumption of nominal solar radiation levels during operation.

### 3.7 SRS Best Practices

To comply with modern standards and effectively convey requirements:

1. **Use Clear and Precise Language:** Write measurable, testable, and unambiguous requirements, avoiding vague terms like "efficient" or "easy to use."
2. **Align with Industry Standards:** Follow frameworks such as ISO/IEC/IEEE 29148:2011 or similar for software requirement definitions.
3. **Visual Aids and Models:** Enhance understanding using diagrams (e.g., UML models, data flow diagrams, or state machines) and prototyping where applicable.
4. **Enable Traceability:** Maintain a requirements traceability matrix (RTM) to map requirements to design, code, and testing activities. Include system hazards, if applicable.
5. **Support Agile, Hybrid, or Traditional Workflows:** Include flexibility in the SRS structure to accommodate iterative development or evolving requirements while maintaining version control.
6. **Use Requirements Management Tools:** Leverage specialized tools (e.g., Jama Connect, DOORS, or JIRA) to manage complex requirements efficiently.
7. **Collaboration and Stakeholder Involvement:** Engage users, developers, testers, and other stakeholders in reviewing the SRS for accuracy, completeness, and alignment with user needs.

Incorporating best practices and modern methodologies ensures the SRS remains a living document that adapts to the dynamic needs of the project while fully supporting the development lifecycle. Keep the SRS consistent, concise, and focused on enabling successful delivery of the software solution.

### 3.8 Additional Guidance

When creating the SRS, take into consideration how Bidirectional Requirements Traceability will be performed and documented as one directly impacts the other. At a minimum, each requirement in the SRS must trace to a parent requirement and a test case. How this trace is done depends on how the documents are organized.

Guidance for bidirectional requirements traceability is found in [SWE-052 - Bidirectional Traceability](#) of this Handbook.

Links to Additional Guidance materials for this subject have been compiled in the Relevant Links table. Click here to see the [Additional Guidance](#) in the Resources tab.

## 4. Small Projects

For small-scale software projects with fewer requirements to document, simplifying and streamlining the process is essential to reduce overhead while maintaining clarity, traceability, and accuracy. The following guidance incorporates modern practices and tools to enhance efficiency, minimize errors, and adapt to the unique needs of small projects. By leveraging available technologies and lightweight workflows, even small projects can create robust and usable Software Requirements Specifications (SRS).

#### 1. Start with Simple and Practical Tools

Small projects typically benefit from low-complexity tools that are easy to maintain and accessible to all team members. Consider the following approaches:

a. **Use Spreadsheets for Requirements Documentation**

A spreadsheet (e.g., Microsoft Excel, Google Sheets) can be an effective tool for organizing, maintaining, and visually tracing requirements for projects with a limited number of requirements. Use this method for projects requiring flexibility and lightweight workflows:

- **Structure:** Include key columns such as Requirement ID, Description, Type (Functional/Non-Functional), Priority, Status, Verification Method, Test Case, and Parent Requirement Traceability. *Example Structure:*

| ID    | Description                      | Type           | Priority | Verification Method | Test Case <sup>1</sup> | Parent Requirement <sup>2</sup> |
|-------|----------------------------------|----------------|----------|---------------------|------------------------|---------------------------------|
| FR-1  | User login requires MFA.         | Functional     | High     | Test (Simulated)    | 5.2.1                  | None                            |
| NFR-2 | System uptime must exceed 99.9%. | Non-Functional | Medium   | Test (Stress)       | 6.3.2                  | 2.3.5.7 System Availability Req |

<sup>1</sup> Integration Test Procedures

<sup>2</sup> System Requirements Specification

- **Benefits:**
  - Simple and widely accessible format.
  - Easy to share and update collaboratively.
  - Compatibility with basic tools like Word processors and plotting software for diagrams.
- **Tips:** Utilize spreadsheet filters, coloring schemes (e.g., green for completed requirements, red for pending), and basic sorting for improved visualization and requirements tracking.

b. **Leverage Lightweight Requirements Management Tools**

If the team has the capacity to use software tools, consider platforms like:

- **Trello or Jira:** Both can track requirements as "tasks" or tickets, visualize workflows (e.g., Kanban boards), and enable team collaboration during iterative phases like Agile development. Use tags or categories to identify requirements type, priority, or dependency relationships.
- **Open-source Tools:** For budget-conscious projects, open-source requirements management tools like **ReqView** or **Traceable Requirements** provide lightweight functionality for requirement storage and traceability.

*Note: Open-source tools used on NASA IT resources must be approved by NASA IT prior to installation and use.*